



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

PYTEST PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Testing

TOTAL: 10 QUESTIONS

Q1 What is PyTest and what problem in Testing does it solve?

Threat Model

Q6 How does PyTest handle reliability and high availability?

Observability

Q2 What are the core components or concepts in PyTest?

Architecture

Q7 What observability does PyTest provide out of the box?

Lifecycle

Q3 How does PyTest compare to its main alternatives?

Configuration

Q8 What are common performance bottlenecks in PyTest?

Compliance

Q4 How do you get started with PyTest for a new project?

Hardening

Q9 What security best practices apply when running PyTest?

Testing

Q5 What configuration options most affect PyTest's behavior in production?

SSO / IdP

Q10 What mistakes do teams commonly make when adopting PyTest?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 PyTest is a tool or platform

Mitigation

PyTest is a tool or platform that automates or simplifies a specific concern in the Testing domain, reducing manual effort and operational complexity for engineering teams.

A6 PyTest provides HA through leader election,

Observability

PyTest provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A2 PyTest has key building blocks that

Primitives

PyTest has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A7 PyTest exposes metrics (Prometheus endpoint

Automation

PyTest exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A3 PyTest trades off simplicity against feature

Hardening

PyTest trades off simplicity against feature richness differently than its alternatives. Choose PyTest when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A8 Performance bottlenecks typically stem from

Governance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A4 Install PyTest via its package manager

Gotchas

Install PyTest via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A9 Run PyTest with the principle of

Assessment

Run PyTest with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A5 Production-critical settings typically include

Federation

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A10 Common pitfalls include skipping the

Optimization

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.